BÁO CÁO THỰC HÀNH

ĐỊNH VỊ AMCL, ĐIỀU HƯỚNG NAV2 VÀ NHẬN DIỆN VẬT THỂ YOLO

TurtleBot 4 Simulation · ROS 2 Humble · Gazebo · RViz2

Thông tin	Nội dung
Sinh viên / MSSV	.....
Lớp / học phần	.....
Giảng viên	.....
Ngày nộp	.....

Tóm tắt kết quả đã xác minh

Sử dụng bản đồ occupancy grid đã lưu để chạy mô phỏng, AMCL và Nav2; chương trình Python gửi tuần tự ba waypoint lấy từ RViz và một điểm quay về (0,0). Log xác nhận cả bốn goal có trạng thái “reached”. Node YOLO nhận ảnh RGB, đổi qua CvBridge, suy luận bằng yolov8n.pt trên CPU, publish ảnh gắn nhãn và ghi lớp/confidence ra terminal.

Phạm vi minh chứng: hình ảnh RViz/Gazebo do người thực hiện cung cấp, ảnh YOLO đã gắn bounding box, map YAML/PGM và log runtime hiện có. Video demo đã được người thực hiện quay nhưng chưa có URL; mục URL video để trống để bổ sung, không tự tạo link.

Bản báo cáo có thể hoàn thiện thông tin cá nhân và đường dẫn ở trang cuối trước khi nộp.

1. Mục tiêu và kiến trúc hệ thống

Bài thực hành kết hợp định vị robot trên bản đồ có sẵn, điều hướng tự động qua các tọa độ trong frame map, đồng thời xử lý luồng camera bằng mô hình YOLO. Các thành phần chính được đóng gói trong tb4_navigation_practice.

Luồng xử lý tổng quát

Gazebo/TurtleBot 4 → /scan, TF, camera RGB → map_server + AMCL → Nav2 planner/controller → robot; camera RGB → CvBridge → YOLO → /yolo/detections/image + log terminal.

Node, topic và action sử dụng

Thành phần	Vai trò / giao tiếp
map_server	Nạp map_with_objects.yaml và ảnh occupancy map_with_objects.pgm; cung cấp bản đồ cho AMCL/RViz.
AMCL	Ước lượng pose trong frame map. Khởi tạo bằng công cụ 2D Pose Estimate; theo dõi /amcl_pose và TF map → odom → base_link.
LiDAR / RViz2	Dữ liệu /scan hiển thị trên occupancy map để kiểm tra scan trùng tường/corridor.
waypoint_navigator	Node Python dùng nav2_simple_commander.BasicNavigator, gửi lần lượt các pose tới action /navigate_to_pose (NavigateToPose).
yolo_detector	Node Python subscribe /oakd/rgb/preview/image_raw; xuất ảnh đã annotate lên /yolo/detections/image.
TurtleBot 4 / Nav2	Nav2 lập kế hoạch và điều khiển robot qua stack điều hướng của TurtleBot 4. Tham số và remap điều khiển được lấy từ launch/config của package.

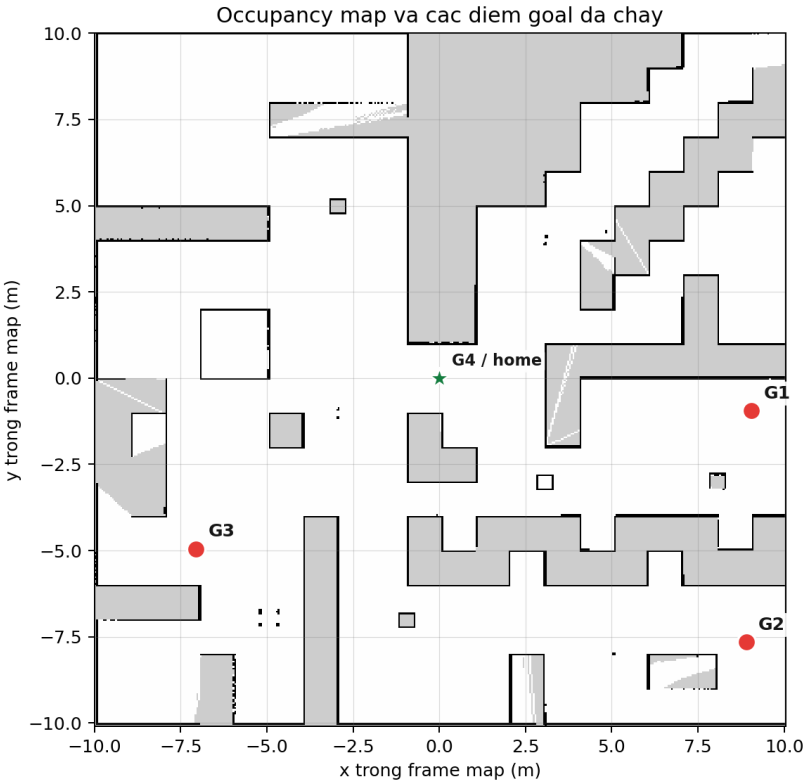
Các ROS interface cốt lõi cần quan sát

Localization: /map, /scan, /initialpose, /amcl_pose, /tf. Navigation: action /navigate_to_pose. Vision: /oakd/rgb/preview/image_raw → /yolo/detections/image.

Tên topic có thể thay đổi theo remap hoặc cấu hình của môi trường; cấu hình YOLO hiện tại đặt trực tiếp hai topic camera và ảnh kết quả như trên.

2. Bản đồ và quy trình định vị

Bản đồ cuối dùng cho localization/navigation là bản đồ quét maze có các vật thể mô phỏng. Hình bên dưới overlay bốn waypoint lên hệ tọa độ map. Các marker chỉ biểu diễn vị trí goal, không phải quỹ đạo thực tế của robot.



Bản đồ sử dụng

map_with_objects.yaml
map_with_objects.pgm

Resolution: 0.05 m/pixel
Kích thước: 401 × 402 pixel
Origin: [-10, -10.1, 0]

Ước lượng pixel: free 70.9%, occupied 3.4%, unknown 25.7%.

Các tỷ lệ được tính từ ảnh PGM với quy ước pixel đen là occupied, trắng là free và vùng xám còn lại là unknown; đây là thống kê ảnh, không phải độ chính xác bản đồ.

Hình 1. Bản đồ occupancy và các tọa độ goal được cấu hình; Goal 4 là điểm quay về (0,0).

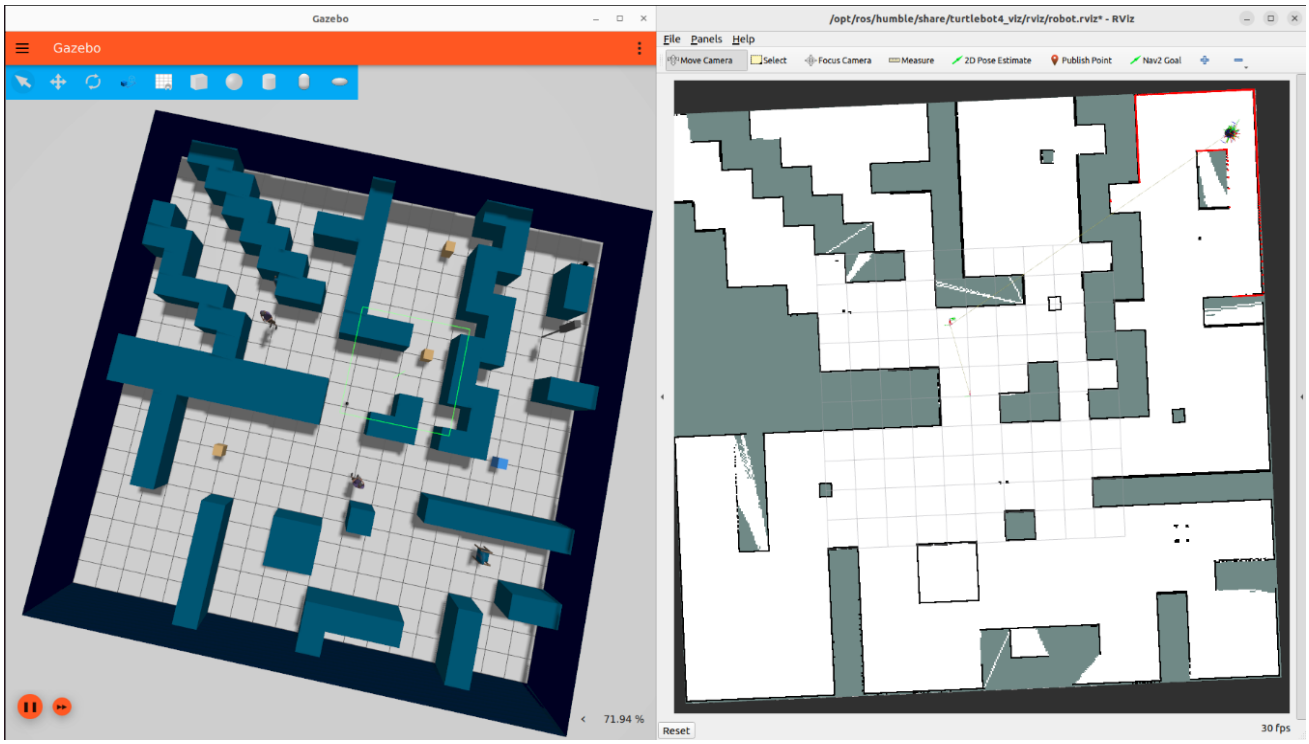
Quy trình localization khi chạy lại

1) Khởi chạy mô phỏng đúng world maze cùng launch localization/navigation và map YAML. 2) Chờ map, AMCL, Nav2 và TF khởi tạo. 3) Dùng RViz2 → 2D Pose Estimate để đặt pose ban đầu theo vị trí robot trong Gazebo. 4) So sánh /scan với tường/corridor trên map; chỉ bắt đầu waypoint khi scan khớp và pose AMCL ổn định.

Ảnh bản đồ map_with_objects được tạo trong quá trình quét có vật thể. Nếu thay đổi bố trí vật thể/world sau khi lưu map, cần quét và lưu lại bản đồ tương ứng trước khi localization.

3. Minh chứng Gazebo và RViz2

Ảnh chụp do người thực hiện cung cấp cho thấy world maze đang chạy trong Gazebo ở cạnh RViz2. Bên RViz hiển thị occupancy map, vị trí robot và các scan/costmap overlay; ảnh phù hợp để minh họa việc nạp map và kiểm tra định vị.



Hình 2. Cửa sổ Gazebo và RViz2 trong quá trình thực hành. Đây là ảnh trạng thái môi trường/bản đồ; kết quả hoàn thành waypoint được xác nhận riêng bằng log điều hướng.

Trước khi quay video nộp bài, nên đảm bảo ảnh/video nhìn rõ ít nhất: bản đồ đã nạp, robot và scan trên RViz2; robot tự di chuyển qua Goal 1 → Goal 2 → Goal 3; cửa sổ hoặc Image display của ảnh YOLO annotated hoạt động song song.

4. Điều hướng tự động bằng Nav2

waypoint_navigator.py tạo pose có frame_id=map và timestamp hiện tại. Hướng yaw (radian) được đổi thành quaternion phẳng z=sin(yaw/2), w=cos(yaw/2). BasicNavigator đợi Nav2 active, gọi goToPose cho từng goal, chờ task kết thúc và dừng chuỗi nếu kết quả khác SUCCEEDED.

```
pose.header.frame_id = 'map'
pose.pose.orientation.z = sin(yaw / 2)
pose.pose.orientation.w = cos(yaw / 2)
navigator.goToPose(pose)
while not navigator.isTaskComplete(): ...
if navigator.getResult() != TaskResult.SUCCEEDED: stop
```

Danh sách tọa độ cấu hình

Goal	x (m)	y (m)	yaw (rad)	Kết quả log
1	9.052786827	-0.947074354	0.0	Reached
2	8.895812988	-7.656434536	0.0	Reached
3	-7.061883450	-4.956677437	0.0	Reached
4 (quay về)	0.0	0.0	0.0	Reached

Kết quả kiểm chứng

Log ROS 2 của lần chạy cuối ghi lần lượt “Goal 1 reached.”, “Goal 2 reached.”, “Goal 3 reached.” và “Goal 4 reached.”, kết thúc bằng “All waypoint goals reached.”. Như vậy minh chứng hiện có xác nhận hoàn tất 4/4 waypoint trong lần chạy được ghi log; các lần chạy lỗi trước đó không được dùng làm kết quả cuối.

Để chạy lại, cần bật mô phỏng/localization/Nav2, xác nhận initial pose và AMCL trước; sau đó mới chạy node. Không dùng RViz Nav2 Goal thủ công trong lúc quay minh chứng tự động.

5. Pipeline nhận diện YOLO

Node yolo_detector subscribe ảnh camera với QoS dành cho sensor. CvBridge đổi sensor_msgs/Image sang OpenCV BGR; YOLOv8n chạy trên CPU, ngưỡng confidence 0.35 và giới hạn inference 2 Hz. Kết quả được vẽ bounding box, đổi ngược thành ROS Image, giữ header nguồn và publish; tên lớp/confidence được ghi ra terminal.

```
image = bridge.imgmsg_to_cv2(msg, desired_encoding='bgr8')
result = model(image, conf=0.35, device='cpu', verbose=False)[0]
annotated = result.plot()
publisher.publish(bridge.cv2_to_imgmsg(annotated, encoding='bgr8'))
logger.info('Detected: class (confidence)')
```

Thông số và topic hiện tại

Input: /oakd/rgb/preview/image_raw Output: /yolo/detections/image
Model: yolov8n.pt (Ultralytics, pretrained COCO) Device: CPU
Confidence: 0.35 Rate: 2 Hz CPU threads: 2
Hiển thị: ảnh annotated publish để xem trên RViz2; OpenCV window mặc định tắt.



Hình 3. Ảnh camera đã được YOLO annotate; ví dụ nhãn “traffic light” với confidence 0.75. Ảnh là dự đoán của mô hình, không tự nó chứng minh ground-truth của vật thể.

Tổng hợp các record lớp trong log YOLO

Nhãn mô hình	Số record trong log	Confidence quan sát
traffic light	1012	0.35 – 0.90
person	271	0.38 – 0.91
airplane	18	0.36 – 0.74
chair	16	0.46 – 0.71
stop sign	6	0.37 – 0.69
refrigerator	3	0.57 – 0.72
bed	2	0.35 – 0.36
bench	2	0.44 – 0.48

Tổng cộng 1330 record dự đoán trong file log. Đây là số lần lớp được ghi qua nhiều frame, không phải số lượng vật thể duy nhất trong world. Log có các nhãn person, chair và traffic light. COCO không có lớp “box” tổng quát; các vật thể không phù hợp lớp huấn luyện có thể bị bỏ sót hoặc phân loại sai.

6. Cách build, chạy và hoàn thiện hồ sơ

Các lệnh dưới đây phản ánh package/workspace hiện có. Thay đường dẫn map bằng đường dẫn tuyệt đối của workspace đang dùng nếu khác.

```
source /opt/ros/humble/setup.bash
cd ~/tb4_ws
colcon build --packages-select tb4_navigation_practice
source install/setup.bash

ros2 launch tb4_navigation_practice tb4_navigation.launch.py \
  world:=maze \
  map:=$HOME/tb4_ws/src/tb4_navigation_practice/maps/map_with_objects.yaml

# RViz2: đặt 2D Pose Estimate, ki□m tra AMCL và /scan trước
ros2 launch tb4_navigation_practice tb4_assignment.launch.py
```

Các file liên quan trong repository

Điều hướng: tb4_navigation_practice/waypoint_navigator.py; config/goals.yaml; launch/tb4_assignment.launch.py.
YOLO: tb4_navigation_practice/yolo_detector.py; config/yolo.yaml.
Map: maps/map_with_objects.yaml và maps/map_with_objects.pgm.
Bringup: launch/tb4_navigation.launch.py; config localization/Nav2; README.md của package.

Hạn chế và lưu ý

• Initial pose AMCL phải được đặt đúng theo Gazebo trước khi gửi goal; nếu map→odom chưa ổn định, Nav2 có thể lập đường sai hoặc thất bại.
• Goal yaw hiện đặt 0 rad vì RViz Publish Point chỉ ghi nhận vị trí; có thể chỉnh yaw trong goals.yaml nếu cần hướng cuối cụ thể.
• YOLO dùng pretrained COCO trên CPU; nhãn là dự đoán có confidence chứ không bảo đảm nhận diện đúng mọi mô hình SDF. Với yêu cầu nhận diện riêng box/chair/person theo ground-truth, nên kiểm tra ảnh thật và cân nhắc huấn luyện model phù hợp.
• Trong cấu hình, giới hạn 2 thread và 2 Hz nhằm giảm tải CPU; mức FPS thực tế phụ thuộc máy và Gazebo.

Kết luận

Các thành phần bắt buộc của bài đã được thực hiện trong môi trường mô phỏng: map được nạp, AMCL/Nav2 được khởi chạy, chương trình điều hướng tự gửi chuỗi goal và YOLO xử lý ảnh camera rồi xuất ảnh annotated. Bằng chứng log xác nhận 4 goal thành công; log YOLO ghi nhận các nhãn person/chair/traffic light. Trước khi nộp, bổ sung thông tin cá nhân và URL video Google Drive đã bật quyền xem công khai.

Liên kết nộp bài

Hạng mục	Đường dẫn / trạng thái
GitHub repository	https://github.com/Quannn5/TB4_nav2 (đã xác minh public qua GitHub API).
Video demo Google Drive	BỔ SUNG URL VIDEO CÔNG KHAI: .....
PDF báo cáo	Tệp hiện tại: docs/bao_cao_nav2_yolo.pdf

Trước khi gửi giảng viên: điền tên/MSSV/lớp; dán link video và thử mở bằng cửa sổ ẩn danh; xác nhận repository hiển thị public và có source, launch, map, config, README.